

Reviewer Pack — Stable-Rank of Layer-Symbol Matrix Separates Read-Twice BPs ...

Entry 46c3da6284c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 09:02:44 UTC

Stable-Rank of Layer-Symbol Matrix Separates Read-Twice BPs for IP₂

Entry ID: 46c3da6284c4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 09:02:44 UTC

1. Conjecture

Field A (mathematical branch): Forster-style stable-rank / numerical-rank theory (Forster 2002; Linial-Shraibman 2009) — a corner of sign-rank / discrepancy-based lower-bound machinery transplanted from communication matrices to the operator-valued symbol bundle of a branching program; this bridge has essentially no presence in BP literature outside the unrelated Nechiporuk rank tradition. **Field B** (complexity object): Read-twice oblivious branching program size for IP₂ on $2n$ adversarial-ordered variables (Borodin-Razborov-Smolensky 1993 open separation), in the meta-complexity sense of seeking a ‘compressibility witness’ $\rho(P)$ with $\rho(P)=O(\log \text{size}(P))$ — a poly-time-computable structural invariant of the BP, NOT of its truth-table.

Statement:

For a read-twice oblivious BP P of size s on $2n$ variables, with layers $t=1..4n$ and transition matrices $M_t(b) \in \{0,1\}^{s \times s}$ for $b \in \{0,1\}$, form the layer-symbol matrix $S(P) := [\Delta_1 \mid \Delta_2 \mid \dots \mid \Delta_{4n}] \in \mathbb{R}^{s \times 4n \cdot s}$ where $\Delta_t := M_t(1) - M_t(0)$. Define the stable-rank invariant $\rho(P) := \|S(P)\|_{F^2} / \|S(P)\|_{op}^2$. Conjecture: (i) $\rho(P) \leq 2 \cdot s$ for every read-twice BP P (numerical-rank anchor: this follows from $\text{rank} \leq s$ but we conjecture the tighter constant 2 holds for all syntactic BPs); (ii) for every read-twice BP P with output correlation $\langle f_P, \text{IP}_2 \rangle \geq \varepsilon$ over uniform $2n$ -bit inputs, $\rho(P) \geq c \cdot \varepsilon^2 \cdot 2^{\Theta(n/2)}$ for an absolute constant $c > 0$. A single instance with $\langle f_P, \text{IP}_2 \rangle \geq \varepsilon$ and $\rho(P) < c \cdot \varepsilon^2 \cdot 2^{\Theta(n/2)}$, or any read-twice BP with $\rho(P) > 2s$, refutes it.

Rationale (proposer’s reasoning):

Each Δ_t records how flipping one read at layer t shifts the BP’s state operator; for a small BP these shifts share an s -dimensional column space, capping the stable rank at $O(s)$. IP₂’s adversarial pairing of x_i with y_i forces the symbol bundle to encode $2^{\Theta(n)}$ mutually-orthogonal cross-variable shift directions (after the middle cut, each pair (x_i, y_i) contributes a Hadamard-flavoured outer product), so the Frobenius mass spreads across exponentially many comparable singular values, making $\|S\|_{F^2} / \|S\|_{op}^2$ grow geometrically with n . The setup avoids natural-proofs (ρ is a structural property of P , not its truth-table), algebrization (singular values are analytic, not ring-arithmetic), and relativization (no oracle).

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e4c44c0eac6fed77

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{3, 4, 5, 6\}$, widths $w \in \{4, 8, 16\}$, and 30 seeds (~ 360 read-twice BPs plus canonical IP₂ BPs), record (s, ρ, ε) per BP. SUPPORT iff every BP satisfies $\rho \leq 2s$ AND every BP with $\varepsilon \geq 0.25$ satisfies $\rho \geq 0.05 \cdot \varepsilon^2 \cdot 2^{\lfloor n/2 \rfloor}$. Any single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forster stable rank branching program lower bound IP₂ inner product - read-twice oblivious branching program inner product mod 2 sign-rank discrepancy - numerical rank transition matrix branching program Linial Shraibman communication

Top relevant hits considered: - [<http://arxiv.org/abs/0710.5926v2>] Mod 2 cohomology of 2-local finite groups of low rank - [<http://arxiv.org/abs/1503.07648v2>] Sign rank versus VC dimension - [<http://arxiv.org/abs/1708.02054v1>] Pseudorandom Bits for Oblivious Branching Programs - [<http://arxiv.org/abs/0809.2093v2>] An approximation algorithm for approximation rank - [<http://arxiv.org/abs/1706.02207v3>] On The Communication Complexity of High-Dimensional Permutations - [<http://arxiv.org/abs/1409.4035v2>] The Corruption Bound, Log Rank, and Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import json
from fractions import Fraction

def matrix_multiply(A, B):
    """Multiply two matrices A and B."""
    if not A or not B or len(A[0]) != len(B):
        raise ValueError("Incompatible matrix dimensions for multiplication")
    result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]
```

```

    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_subtract(A, B):
    """Subtract matrix B from matrix A."""
    if len(A) != len(B) or len(A[0]) != len(B[0]):
        raise ValueError("Matrices must have the same dimensions for subtraction")
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_transpose(A):
    """Transpose a matrix A."""
    return [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]

def matrix_norm_frobenius(A):
    """Compute the Frobenius norm of matrix A."""
    return math.sqrt(sum(sum(a ** 2 for a in row) for row in A))

def matrix_norm_op(A):
    """Compute the operator norm (spectral norm) of matrix A."""
    # Using the power iteration method to approximate the largest singular value
    if not A or not A[0]:
        return 0.0
    b_k = [random.random() for _ in range(len(A[0]))]
    for _ in range(100):
        # Multiply A and b_k
        b_k = [sum(A[i][j] * b_k[j] for j in range(len(b_k))) for i in range(len(A))]
        # Normalize b_k
        norm = math.sqrt(sum(x ** 2 for x in b_k))
        if norm == 0:
            break
        b_k = [x / norm for x in b_k]
    # Compute A*b_k
    Ab_k = [sum(A[i][j] * b_k[j] for j in range(len(b_k))) for i in range(len(A))]
    # Compute the norm of A*b_k
    norm_Ab_k = math.sqrt(sum(x ** 2 for x in Ab_k))
    return norm_Ab_k

def compute_stable_rank(S):
    """Compute the stable rank of matrix S."""
    if not S or not S[0]:
        return 0.0
    frobenius_norm = matrix_norm_frobenius(S)
    op_norm = matrix_norm_op(S)
    if op_norm == 0:
        return 0.0
    return frobenius_norm ** 2 / op_norm ** 2

def generate_random_bp(n, w, seed):
    """Generate a random read-twice BP of size w and n variables."""
    random.seed(seed)
    s = w
    layers = 4 * n
    M = []

```

```

for _ in range(layers):
    M_t_0 = [[random.randint(0, 1) for _ in range(s)] for _ in range(s)]
    M_t_1 = [[random.randint(0, 1) for _ in range(s)] for _ in range(s)]
    M.append((M_t_0, M_t_1))
return M, s

def construct_symbol_matrix(M, s):
    """Construct the symbol matrix S(P) from the transition matrices M."""
    layers = len(M)
    S = []
    for M_t_0, M_t_1 in M:
        Delta_t = matrix_subtract(M_t_1, M_t_0)
        S.extend(Delta_t)
    # Reshape S into an s x (4n * s) matrix
    reshaped_S = []
    for i in range(s):
        row = []
        for j in range(0, len(S), s):
            row.extend(S[j + i])
        reshaped_S.append(row)
    return reshaped_S

def evaluate_bp(M, s, n):
    """Evaluate the BP on all  $2^{2n}$  inputs."""
    inputs = []
    for i in range(2 ** (2 * n)):
        inputs.append([(i >> j) & 1 for j in range(2 * n)])
    outputs = []
    for input_bits in inputs:
        state = [1 if i == 0 else 0 for i in range(s)]
        for t in range(4 * n):
            M_t_0, M_t_1 = M[t]
            b = input_bits[t % (2 * n)]
            if b == 0:
                M_t = M_t_0
            else:
                M_t = M_t_1
            new_state = [0] * s
            for i in range(s):
                for j in range(s):
                    new_state[i] += state[j] * M_t[j][i]
            state = new_state
        outputs.append(state[0])
    return outputs

def compute_correlation(outputs, n):
    """Compute the correlation of the BP outputs with IP_2."""
    ip2_outputs = []
    for i in range(2 ** (2 * n)):
        x = [(i >> j) & 1 for j in range(n)]
        y = [(i >> (n + j)) & 1 for j in range(n)]
        ip2_outputs.append(1 if sum(x[j] & y[j] for j in range(n)) % 2 == 0 else -1)
    correlation = sum(outputs[i] * ip2_outputs[i] for i in range(len(outputs))) / len(outputs)
    return correlation

def run_trial(seed):

```

```

"""Run a single trial with the given seed."""
random.seed(seed)
n_values = [3, 4, 5, 6]
w_values = [4, 8, 16]
instances_tested = 0
conjecture_holds = True
counterexample = ""
metric_values = []

for n in n_values:
    for w in w_values:
        M, s = generate_random_bp(n, w, seed)
        S = construct_symbol_matrix(M, s)
        rho = compute_stable_rank(S)
        outputs = evaluate_bp(M, s, n)
        epsilon = compute_correlation(outputs, n)
        metric_values.append(rho)

        if rho > 2 * s:
            conjecture_holds = False
            counterexample = f"BP with rho={rho} > 2s={2*s}"
            break

        if epsilon >= 0.25 and rho < 0.05 * epsilon ** 2 * (2 ** (n / 2)):
            conjecture_holds = False
            counterexample = f"BP with epsilon={epsilon} >= 0.25 and rho={rho} < 0.05 * epsilon"
            break

        instances_tested += 1

    if not conjecture_holds:
        break

if conjecture_holds:
    # Test the canonical IP_2 BP
    n = 3 # Smallest n for IP_2
    s = 2 ** n
    M = []
    for t in range(4 * n):
        M_t_0 = [[0 for _ in range(s)] for _ in range(s)]
        M_t_1 = [[0 for _ in range(s)] for _ in range(s)]
        for i in range(s):
            for j in range(s):
                if t < 2 * n:
                    if (i >> t) & 1 == (j >> t) & 1:
                        M_t_0[i][j] = 1
                        M_t_1[i][j] = 1
                else:
                    if (i >> (t - 2 * n)) & 1 == (j >> (t - 2 * n)) & 1:
                        M_t_0[i][j] = 1
                        M_t_1[i][j] = 1
            M.append((M_t_0, M_t_1))
    S = construct_symbol_matrix(M, s)
    rho = compute_stable_rank(S)
    outputs = evaluate_bp(M, s, n)
    epsilon = compute_correlation(outputs, n)
    metric_values.append(rho)

```

```

    if rho > 2 * s:
        conjecture_holds = False
        counterexample = f"IP_2 BP with rho={rho} > 2s={2*s}"
    if epsilon >= 0.25 and rho < 0.05 * epsilon ** 2 * (2 ** (n / 2)):
        conjecture_holds = False
        counterexample = f"IP_2 BP with epsilon={epsilon} >= 0.25 and rho={rho} < 0.05 * epsilon"
    instances_tested += 1

mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0.0
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))

return {
    "metric_name": "stable_rank",
    "metric_value": mean_metric,
    "metric_std": std_metric,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={seed}")
                break

```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	26.500000025265745	□	BP with rho=26.500000025265745 > 2s=8
23	91.63993200010147	□	BP with rho=91.63993200010147 > 2s=8
37	58.39948414033939	□	BP with rho=58.39948414033939 > 2s=8
53	84.0	□	BP with rho=84.0 > 2s=8

Seed	Metric value	Holds?	Counterexample
71	129.9529762678772	□	BP with rho=129.9529762678772 > 2s=8
89	27.934945568278728	□	BP with rho=27.934945568278728 > 2s=8
103	93.0	□	BP with rho=93.0 > 2s=8
127	32.84735895260694	□	BP with rho=32.84735895260694 > 2s=8
149	94.021249128744	□	BP with rho=94.021249128744 > 2s=8
167	48.874613781593986	□	BP with rho=48.874613781593986 > 2s=8
191	26.40122847652249	□	BP with rho=26.40122847652249 > 2s=8
211	92.14670376266143	□	BP with rho=92.14670376266143 > 2s=8
233	35.140873035009676	□	BP with rho=35.140873035009676 > 2s=8
257	27.194992312031793	□	BP with rho=27.194992312031793 > 2s=8
277	23.85861249086548	□	BP with rho=23.85861249086548 > 2s=8
311	100.47391599276868	□	BP with rho=100.47391599276868 > 2s=8
347	37.0507030912602	□	BP with rho=37.0507030912602 > 2s=8
389	37.050703091260196	□	BP with rho=37.050703091260196 > 2s=8
421	22.057551269602595	□	BP with rho=22.057551269602595 > 2s=8
463	87.000000000000001	□	BP with rho=87.000000000000001 > 2s=8
503	80.24174837076876	□	BP with rho=80.24174837076876 > 2s=8
547	39.25023292246046	□	BP with rho=39.25023292246046 > 2s=8

Seed	Metric value	Holds?	Counterexample
593	47.12182530062137	□	BP with $\rho=47.12182530062137 > 2s=8$
631	65.81197395850668	□	BP with $\rho=65.81197395850668 > 2s=8$
677	0.0	□	BP with $\epsilon=176.703125 \geq 0.25$ and $\rho=0.0 < 0.05 * \epsilon^2 * 2^{(n/2)}$
727	36.561209216717984	□	BP with $\rho=36.561209216717984 > 2s=8$
773	28.50532667271179	□	BP with $\rho=28.50532667271179 > 2s=8$
821	59.148813421486146	□	BP with $\rho=59.148813421486146 > 2s=8$
877	118.51617177379977	□	BP with $\rho=118.51617177379977 > 2s=8$
929	31.94484597356763	□	BP with $\rho=31.94484597356763 > 2s=8$

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	56.088266366581024
metric_std	33.055761244605534
metric_ci95_half	12.070257392783663
metric_min	0.0
metric_max	129.9529762678772
support_fraction	0.0

7. Test stdout (last 2KB)

```

551269602595 > 2s=8", "seed": "421"}
TRIAL: {"metric_name": "stable_rank", "metric_value": 87.00000000000001, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 80.24174837076876, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 39.25023292246046, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 47.12182530062137, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 65.81197395850668, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 36.561209216717984, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 28.50532667271179, "metric_std": 0.0, "instances_tested": 0,
TRIAL: {"metric_name": "stable_rank", "metric_value": 59.148813421486146, "metric_std": 0.0, "instances_tested": 0,

```


8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

Metric-definition / construction-gap bug. By elementary linear algebra, for any matrix A the stable rank $\|A\|_F^2/\|A\|_{op}^2$ is bounded by $\text{rank}(A)$. Since $S(P) \in \mathbb{R}^{s \times 4n \cdot s}$ has $\text{rank} \leq s$, the true $\rho(P)$ must satisfy $\rho(P) \leq s \leq 2s$ — yet the test reports ρ values like 26–129 with $2s=8$, which is mathematically impossible. The reported ‘refutations’ of part (i) therefore cannot be computing $\|S\|_F^2/\|S\|_{op}^2$ on the concatenated S ; likely the implementation sums per-layer ratios $\sum_t \|\Delta_t\|_F^2/\|\Delta_t\|_{op}^2$ (which

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The critic correctly notes that $\rho(P)=\|S\|_F^2/\|S\|_{op}^2$ is mathematically bounded by $\text{rank}(S) \leq s$, so reported values like 26–129 with $2s=8$ are impossible and indicate a metric-implementation bug (likely summing per-layer ratios instead of computing the stable rank of the concatenated matrix). The pre-registered support condition is therefore not validly evaluated, and the ‘falsification’ cannot be trusted. | next: Fix the ρ computation to actually compute $\|S\|_F^2/\|S\|_{op}^2$ on the concatenated $S \in \mathbb{R}^{s \times 4n}$

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	187638
2	preregistration	claude_max	opus	0	0	6419
3	novelty	claude_max	opus	0	0	3575
4	novelty	claude_max	opus	0	0	8895
5	test_gen	mistral	codestral-latest	0	0	318540
6	critic	claude_max	opus	0	0	24311
7	judge	claude_max	opus	0	0	5828

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 555207 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 1}

(full prompt+response transcripts available in `research/audit/46c3da6284c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46c3da6284c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/46c3da6284c4.tar.gz` (if generated)